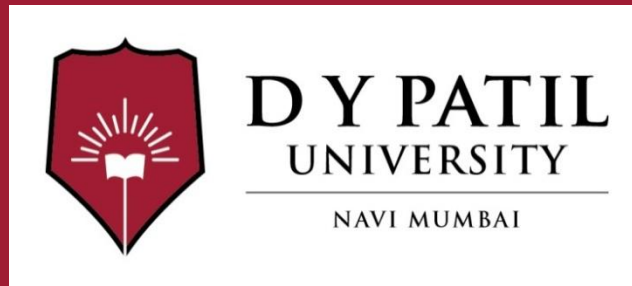


Subject Name :

Unit No: 6

Unit Name : Web Development

Faculty Name :Prachi A Surve



Unit No.: 6

Faculty Name :Prachi A Surve

Index

Lecture No & Topic Name	Slide No
Web Development	
Intro with Flask: Flask setup/installation;	
routing (@app.route);	
templates (Jinja2);	
forms (GET/POST);	
integrate MySQL CRUD via mysql-connector;	
simple app (TODO list/student DB viewer).	
Deploy demo (Heroku/local)	



Unit No:1

Lecture No: 1



Flask

- Flask is a lightweight web framework for Python used to build web applications and APIs.
- It follows a minimal design and provides core features like routing, request handling, and template rendering while allowing developers to add extensions as needed.
- It is widely used for building small to medium web applications due to its simplicity and flexibility.
- It was developed by Armin Ronacher.
- Flask's framework is more explicit than [Django's framework](#) and is also easier to learn because it has less base code to implement a simple web application.
- Flask Python is based on the WSGI(Web Server Gateway Interface) toolkit and Jinja2 template engine.

Advantages of Flask

- Flask is a **lightweight** backend framework with minimal dependencies.
- Flask is **easy to learn** because its simple and intuitive API makes it easy to learn and use for beginners.
- Flask is a **flexible Framework** because it allows you to customize and extend the framework to suit your needs easily.
- Flask can be used with **any database** like:- SQL and NoSQL and with **any Frontend Technology** such as [React](#) or [Angular](#).
- Flask is **great for small to medium projects** that do not require the complexity of a large framework.
- Flask Documentation.



Getting Started With Flask

Python3 is required for the installation of the Python Web Framework Flask. You can start by importing Flask from the Flask Python package on any Python IDE. For installation on any environment, you can execute the command "**pip install flask**" on your terminal. Let's look at an example of a basic flask app.

```
from flask import Flask
app = Flask(__name__) # Flask constructor

# A decorator used to tell the application
# which URL is associated function
@app.route('/')
def hello():
    return 'HELLO'

if __name__ == '__main__':
    app.run(debug=True)
```

Explanation:

- **Flask(__name__)**: Creates the Flask app.
- **@app.route('/')**: Defines the home route (/).
- **def hello()**: creates a function that is bound with '/' route and returns "HELLO" when the root page is accessed.
- **app.run(debug=True)**: runs the app in debug mode. It ensure that app is not need to restart manually if any changes are made in code.



Build Flask Routes in Python

Web frameworks provide routing technique so that user can remember the URLs. It is useful to access the web page directly without navigating from the Home page. It is done through the following `route()` decorator, to bind the URL to a function.

```
# decorator to route URL
@app.route('/hello')

# binding to the function of route
def hello_world():
    return 'hello world'
```

Explanation: If a user visits **http://localhost:5000/hello** URL, the output of the **hello_world()** function will be rendered in the browser.

One alternate way of doing this is by using "`add_url_rule()`" function of an application object, it can also be used to bind URL with the function similar to the above example.

```
def hello_world():
    return 'hello world'

app.add_url_rule('/', 'hello', hello_world)
```



Variables in Flask

Variables in flask are used to build a URL dynamically by adding the variable parts to the rule parameter. It is passed as keyword argument. Here's an example.

```
from flask import Flask
app = Flask(__name__)

@app.route('/hello/<name>')
def hello_name(name):
    return 'Hello %s!' % name

if __name__ == '__main__':
    app.run(debug = True)
```

Check for infinite loading (common issue)

If it's still loading forever:

Possible causes: Port already in use

Another app might be using port 5000.

Fix:

Change port: `app.run(debug=True, port=5001)`

Unit4.py - C:/Users/Student/Unit4.py (3.14.0)

File Edit Format Run Options Window Help

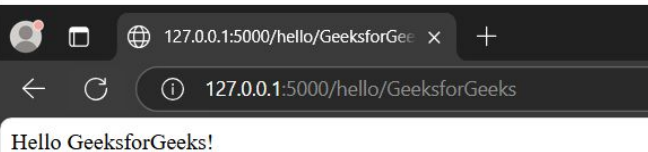
```
from flask import Flask
app = Flask(__name__)

@app.route('/hello/<name>')
def hello_name(name):
    return 'Hello %s!' % name

if __name__ == '__main__':
    app.run(debug=True, port=5001)
```

127.0.0.1:5001/hello/MCA%20Batch%202026

Hello MCA Batch 2026!



D Y PATIL
UNIVERSITY
NAVI MUMBAI

Execution Flow (When You Run the Script)

Step 1:

You run the file:
`python Uni6.py`

Step 2:

- Flask app is created.
- Routes are registered.

Step 3:

- Development server starts (default: `http://127.0.0.1:5000/`).

When a User Sends a Request

Example:

`http://127.0.0.1:5000/hello/MCA Batch 2026`

Step-by-step:

1. Browser sends request to `/hello/MCA Batch 2026`.
2. Flask matches the URL with: `/hello/<name>`
3. Extracts:
`name = "MCA Batch 2026"`
4. Calls:
`hello_name("MCA Batch 2026")`
5. Function returns:
`Hello MCA Batch 2026!`
6. Flask sends this response back to the browser.

Final Output in Browser

Hello MCA Batch 2026!



Flask App Routing

Flask App Routing is the process of mapping URLs to specific functions in a web application.

When a user enters a URL in the browser,

Flask checks the defined routes and executes the corresponding function to generate a response.

This is done using the `@app.route()` decorator, which binds a URL path to a function.

Example: Here, we define two routes: `/` for the homepage and `/hello` for displaying a welcome message. Each route is linked to a function using the `@app.route()` decorator.

```
from flask import Flask
app = Flask(__name__)

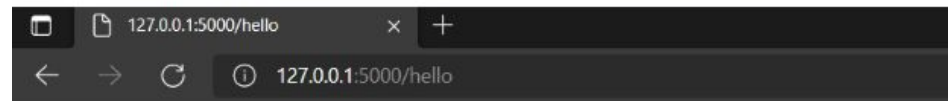
@app.route("/hello")
def hello():
    return "Hello, Welcome to GeeksForGeeks"

@app.route("/")
def index():
    return "Homepage of GeeksForGeeks"

if __name__ == "__main__":
    app.run(debug=True)
```

python filename.py

Output: Open the browser and visit `127.0.0.1:5000/hello`, you will see the following output.



Hello, Welcome to GeeksForGeeks



1. Importing Flask

from flask import Flask

- This line imports the **Flask class** from the Flask library.
- Flask is used to create web applications in Python.

2. Creating the app object

app = Flask(__name__)

- This creates a Flask application instance.
- `__name__` tells Flask where the app is located (helps it find files like templates).

3. Defining a route: `/hello`

@app.route('/hello')

def hello():

 return "Hello, welcome to GeeksForGeeks"

- `@app.route('/hello')` is a **decorator**.
- It tells Flask:
 “When someone visits `/hello`, run this function.”
- The function `hello()`:
 - Executes when `/hello` URL is opened.
 - Returns a string → shown in the browser.

4. Defining the homepage route: `/`

@app.route('/')

def index():

 return "Homepage of GeeksForGeeks"

- `/` represents the **home page**.
- When user opens `http://127.0.0.1:5000/`:
 - `index()` function runs.
 - Displays: `"Homepage of GeeksForGeeks"`

5. Running the app

if __name__ == "__main__":

 app.run(debug=True)

- This ensures the app runs **only when the file is executed directly**.
- `app.run()` starts the web server.
- `debug=True`:
 - Enables **debug mode**
 - Automatically reloads on code changes
 - Shows errors in detail (useful for development)



Advanced Routing in Flask

1. Introduction

Routing in Flask maps a **URL (endpoint)** to a **Python function (view)**.

Advanced routing allows developers to build dynamic, flexible, and modular web applications.

2. Variable Rules (Dynamic Routing)

Flask allows passing variables through the URL.

```
@app.route('/user/<name>')
def user(name):
    return f"Hello {name}"
```

With Type Conversion

```
@app.route('/post/<int:id>')
def post(id):
    return f"Post ID: {id}"
```

Converters:

- **int** → integer
- **float** → decimal
- **string** → default
- **path** → accepts /

3. Multiple Routes for Same Function

```
@app.route('/')
@app.route('/home')
def home():
    return "Welcome"
```

One function handles multiple URLs.

4. HTTP Methods in Routing

```
@app.route('/login', methods=['GET', 'POST'])
def login():
    if request.method == 'POST':
        return "Form Submitted"
    return "Login Page"
```

Controls how requests are handled.

5. URL Building (Dynamic URL Generation)

Using `url_for()` instead of hardcoding URLs:
from flask import url_for

```
url_for('user', name='Prachi')
```

Generates: `/user/Prachi`

Advantages:

- Avoids hardcoding
- Easier maintenance

6. Redirects

from flask import redirect, url_for

```
@app.route('/admin')
def admin():
    return redirect(url_for('login'))
```

Redirects user to another route.

7. Default Values in Routes

```
@app.route('/page/', defaults={'num': 1})
@app.route('/page/<int:num>')
def page(num):
    return f"Page {num}"
```

Handles optional parameters.

8. Error Handling (Custom Routes)

```
@app.errorhandler(404)
def not_found(error):
    return "Page not found", 404
```

Custom error pages improve user experience.



Dynamic URLs

We can also build dynamic URLs by using variables in the URL. To add variables to URLs, use the `<variable_name>` syntax in the route. The function then receives the `<variable_name>` as keyword argument.

Example: In this example, a dynamic route is created where the username is passed through the URL and displayed in the browser.

```
from flask import Flask

app = Flask(__name__)

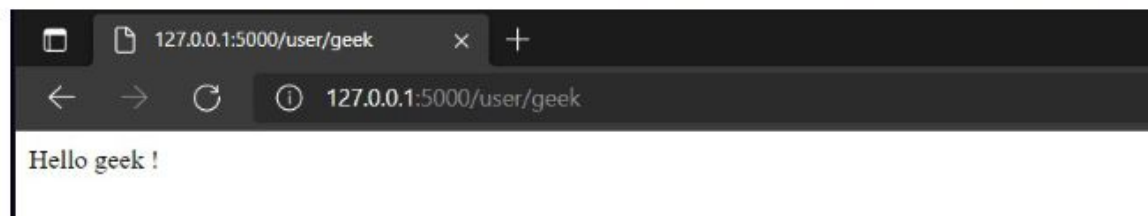
@app.route('/user/<username>')
def show_user(username):
    return f'Hello {username} !'

@app.route("/hello")
def hello():
    return "Hello, Welcome to GeeksForGeeks"

@app.route("/")
def index():
    return "Homepage of GeeksForGeeks"

if __name__ == "__main__":
    app.run(debug=True)
```

Output: Open the browser and visit `127.0.0.1:5000/user/geek`, you will see the following output.



1. Import Flask

from flask import Flask

- Imports the Flask framework to build the web app.

2. Create Flask app

```
app = Flask(__name__)
```

- Creates the main application object.
- `__name__` helps Flask locate resources (files, templates).

3. Dynamic Route (Important part)

```
@app.route('/user/<username>')
def show_user(username):
    return f"Hello {username} !"
```

What's happening:

- `/user/<username>` → **dynamic URL**
- `<username>` is a **variable** in the URL.

Flow:

1. User enters:
`http://127.0.0.1:5000/user/prachi`
2. Flask extracts:
 - `username = "prachi"`
3. Calls function:
 - `show_user("prachi")`
4. Output shown: **Hello prachi !**

4. Static Route: `/hello`

```
@app.route('/hello')
def hello():
    return "Hello, welcome to GeeksForGeeks"
```

- This is a **fixed route** (not dynamic).
- When user visits `/hello`, it shows a welcome message.

5. Homepage Route: `/`

```
@app.route('/')
def index():
    return "Homepage of GeeksForGeeks"
```

- `/` is the **default homepage**.
- Displays the homepage message.

6. Run the app

```
if __name__ == "__main__":
    app.run(debug=True)
```



Additionally, we can also use a converter to convert the variable to a specific data type.

By default, variables are treated as strings.

To specify a data type, use <converter:variable_name>.

The following converters are supported:

- **string:** default type and it accepts any text without a slash.
- **int:** accepts positive integers.
- **float:** accepts positive floating-point values.
- **path:** a string but also accepts slashes.
- **uuid:** accepts UUID strings.



Example: Consider the following example to demonstrate the converter type.

```
from flask import Flask

app = Flask(__name__)

@app.route('/post/<int:id>')
def show_post(id):
    return f'This post has the id {id}'

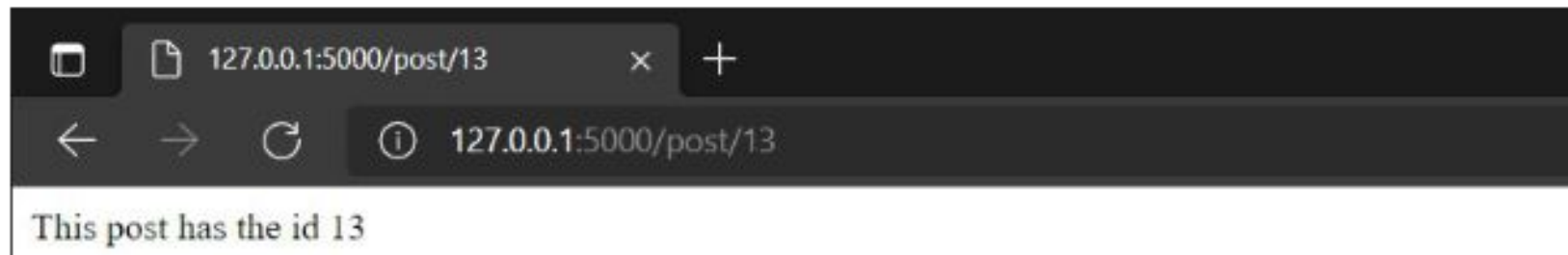
@app.route('/user/<username>')
def show_user(username):
    return f'Hello {username} !'

@app.route("/hello")
def hello():
    return "Hello, Welcome to GeeksForGeeks"

@app.route("/")
def index():
    return "Homepage of GeeksForGeeks"

if __name__ == "__main__":
    app.run(debug=True)
```

Output: Open the browser and visit 127.0.0.1:5000/post/13, you will see the following output.



1. Import Flask

from flask import Flask

- Imports Flask framework to build the web application.

2. Create the app

app = Flask(__name__)

- Initializes the Flask app.
- `__name__` helps Flask find resources.

3. Route with Converter Type (<int:id>)

@app.route('/post/<int:id>')

def show_post(id):

return f'This post has the id {id}'

Key Concept: Converter Type

- `<int:id>` means:
 - Only **integer values** are allowed.
 - Flask automatically converts the value to an **integer**.

Flow:

- User enters:
`http://127.0.0.1:5000/post/13`
- Flask:
 - Matches route `/post/<int:id>`
 - Converts `"13"` → `13` (integer)
- Calls:
 - `show_post(13)`
- Output:
This post has the id 13

Invalid case:

- `/post/abc` → Error (because `abc` is not an integer)

4. Dynamic Route (<username>)

@app.route('/user/<username>')

def show_user(username):

return f'Hello {username} !'

What happens:

- Accepts **any string** as username.

Example:

- `/user/prachi` → **Hello prachi !**

5. Static Route /hello

@app.route('/hello')

def hello():

return "Hello, welcome to GeeksForGeeks"

- Fixed URL.
- Shows a welcome message.

6. Homepage /

@app.route('/')

def index():

return "Homepage of GeeksForGeeks"

- Default page when opening the app.

7. Run the app

if __name__ == "__main__":

app.run(debug=True)

Meaning:

- Runs the server.
- `debug=True`:
 - Auto reloads code
 - Shows detailed errors



add_url_rule() function

The URL mapping can also be done using the `add_url_rule()` function.

This approach is mainly used when the view function is defined in another module.

In fact, the `app.route` calls this function internally.

Syntax:

```
add_url_rule(rule, endpoint, view_func)
```

Example: In the below example, the `show_user` function is mapped to a URL using `add_url_rule()` instead of a decorator.

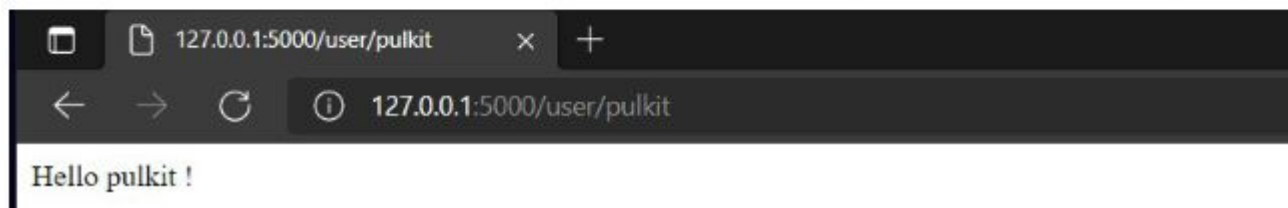
```
from flask import Flask
app = Flask(__name__)

def show_user(username):
    return f'Hello {username} !'

app.add_url_rule('/user/<username>', 'show_user', show_user)

if __name__ == "__main__":
    app.run(debug=True)
```

Output: Open the browser and visit `127.0.0.1:5000/user/pulkit`, you will see the following output.



4. Add URL rule manually

```
app.add_url_rule('/user/<username>', 'show_user', show_user)
```

What this does:

This is an **alternative to `@app.route()` decorator**.

It connects:

- URL → `/user/<username>`
- Function → `show_user`

Parameters explained:

- `/user/<username>` → dynamic URL
- `'show_user'` → endpoint name (identifier)
- `show_user` → function to execute

Flow:

1. User enters:
`http://127.0.0.1:5000/user/pulkit`
2. Flask:
 - Extracts `username = "pulkit"`
3. Calls:
 - `show_user("pulkit")`
4. Output:
Hello pulkit !

1. Import Flask

```
from flask import Flask
```

- Imports the Flask framework to build a web application.

2. Create the app object

```
app = Flask(__name__)
```

- Initializes your Flask app.
- `__name__` helps Flask know where the app is located.

3. Define a function

```
def show_user(username):  
    return f'Hello {username} !'
```

- This function takes a parameter `username`.
- It returns a **dynamic message** using that name.

5. Run the app

```
if __name__ == "__main__":  
    app.run(debug=True)
```



D Y PATIL
UNIVERSITY
NAVI MUMBAI

Flask Rendering Templates

Flask is a lightweight Python web framework that enables developers to build web applications easily. One of its key features is template rendering, which allows dynamic content generation using Jinja2 templating.

We'll explore how to render templates in Flask.

Setting up Flask

Setting up a new flask app requires creating and activating a virtual environment and installing flask to it.

Creating virtual environment

Use the command below to create a new virtual environment in recent Python versions.

```
python -m venv venv
```

Activate the virtual environment

For Windows:

```
venv\Scripts\activate
```



Rendering HTML Templates

One of Flask's powerful features is its ability to render HTML templates using Jinja2.

Instead of returning plain strings in routes, we can use `render_template()` to serve HTML files dynamically.

Create a templates Folder

Flask looks for HTML files in a special directory called templates.

Create a folder named templates in your project directory and create the index.html file.

index.html:

```
<!DOCTYPE html>
<html>
<head>
  <title>Flask App</title>
</head>
<body>
  <h2>Welcome to Flask</h2>
  <p>This is a basic template rendering example.</p>
</body>
</html>
```

Explanation:

Step 1: `<!DOCTYPE html>`

- Declares that the document is an HTML5 file.

Step 2: `<html>`

- Root element that contains the entire webpage.

Step 3: `<head>`

- Contains metadata (not visible on the page).

Step 4: `<title>`

- Sets the title of the webpage (shown in browser tab as **Flask App**).

Step 5: `<body>`

- Contains visible content of the webpage.

Step 6: `<h2>`

- Displays heading: **"Welcome to Flask"**

Step 7: `<p>`

- Displays paragraph text.



Modify app.py to Render the Template

Updating app.py to use `render_template()`:

```
from flask import Flask, render_template

app = Flask(__name__)

@app.route("/")
def index():
    return render_template("index.html")

if __name__ == "__main__":
    app.run()
```

Explanation:

- `@app.route("/")`: Maps the root URL (/) to the `index()` function.
- `render_template("index.html")`: Renders the HTML file.

Explanation:

Step 1: Import Flask

from flask import Flask, render_template

- `Flask`: used to create the web app
- `render_template`: used to load HTML files

Step 2: Create Flask App

app = Flask(__name__)

- Initializes the Flask application
- `__name__` tells Flask where to look for files

Step 3: Define Route

@app.route("/")

- This connects the URL / (homepage) to a function
- When user visits <http://localhost:5000/>, this runs

Step 4: Create Function

def index():

- Function that runs when homepage is accessed

Step 5: Render HTML

return render_template("index.html")

- Loads and displays the `index.html` file
- Flask looks for this file inside a **templates folder**

Step 6: Run the App

if __name__ == "__main__":

app.run()

- Ensures the app runs only when file is executed directly
- Starts the server (default: <http://127.0.0.1:5000/>)



Templating With Jinja2 in Flask

Now, we'll create a new route for demonstrating the usage of the [Jinja template](#) and add it in **app.py**.

```
@app.route("/<name>")
def welcome(name):
    return render_template("welcome.html", name=name)
```

We create a route `"/<name>"` linked to the `welcome` function. The `<name>` part captures any value after `/` and passes it as a parameter to `render_template()`. This makes the variable accessible in the template for rendering or modification.

We create a route `"/<name>"` linked to the `welcome` function. The `<name>` part captures any value after `/` and passes it as a parameter to `render_template()`. This makes the variable accessible in the template for rendering or modification.

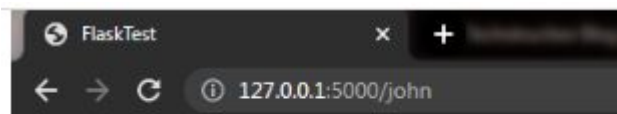
Create a **welcome.html** file inside the templates folder with the following markup.

```
<!DOCTYPE html>
<html>

<head>
  <title>FlaskTest</title>
</head>

<body>
  <h2>Welcome To GFG</h2>
  <h3>Welcome, {{name}}</h3>
</body>

</html>
```



Welcome To GFG

Welcome, john



Explanation:

Step 1: `<!DOCTYPE html>`

- Declares HTML5 document.

Step 2: `<title>`

- Sets browser tab title as **FlaskTest**.

Step 3: `<h2>`

- Static heading: **Welcome To GFG**.

Step 4: `{{name}}` (Important)

- This is a **Jinja2 template variable**.
- It acts as a placeholder.
- Flask replaces `{{name}}` with actual data sent from Python.

👉 Example:

- If `name = "john"` → Output becomes:

Welcome, john

2. Flask Backend (app.py logic)

Even though not fully shown, the working code is typically:

```
from flask import Flask, render_template
```

```
app = Flask(__name__)
```

```
@app.route("/<name>")
```

```
def home(name):
```

```
    return render_template("index.html",  
                           name=name)
```

```
if __name__ == "__main__":
```

```
    app.run()
```

Explanation:

Step 1: Dynamic Route

```
@app.route("/<name>")
```

- `<name>` is a **URL parameter**.
- It allows user input directly from URL.

👉 Example:

- Visiting: `http://127.0.0.1:5000/john`
- Flask captures `"john"` as value of `name`

Step 2: Function Parameter

```
def home(name):
```

- The value from URL is passed to the function.

Step 3: Pass Data to HTML

```
return render_template("index.html", name=name)
```

- Sends `name` to the HTML template.
- Jinja replaces `{{name}}` with this value.

3. How Output is Generated

Step-by-step flow:

1. User enters URL:

`http://127.0.0.1:5000/john`

2. Flask matches route:

`/<name>`

3. `name = "john"`

4. Flask calls:

`home("john")`

5. Template is rendered:

Welcome, john

4. Final Output on Browser

Welcome To GFG

Welcome, john

5. Key Concept Summary

- `{{ }}` → Used to display variables in HTML
- `<name>` → Dynamic URL input
- `render_template()` → Sends data to HTML



Getting started with Jinja Template:

Jinja is a text rendering engine for Python programming language.

It has first-class support among popular Python frameworks like Django and Flask and is used extensively.

It is popular for its easy and straightforward syntax and variety of features including (but not limited to) -

- Variable access.
- Control structures (Conditionals, loops).
- Macros (similar to functions in programming languages).
- Filters
- Inheritance, etc.



Rendering text from Jinja Template using Python

Rendering a jinja template always results in some text (obviously, since this is what jinja is all about as stated earlier).

Rendering a template using a python file involves the following steps -

- *Step 1: Import the necessary libraries and components (objects, functions, etc.) from libraries.*
- *Step 2: Create a jinja rendering environment and store it in a variable. This environment will be used in further steps.*
- *Step 3: Load the template in a variable.*
- *Step 4: Render the template using <template-object>.render() function to obtain text.*
- *Step 5: Print the rendered text to the screen or a file as suitable.*



Basics of Jinja Syntax

Now that we are familiar with the basics of rendering a jinja template and passing data to it with a python program, we proceed to the basic syntax of the jinja templates and how to use the data passed in the template.

Delimiters

All the jinja syntax goes inside the delimiters inside the template.

All the text outside the delimiters is rendered as it is.

There are following three types of delimiters by default -

- `{{ }}` for expressions.
- `{# #}` for comments (even multiline) inside the template.
- `{% %}` for jinja statements (like loops, etc.)

Expressions

We can write expressions inside the Jinja templates in "`{{ }}`" delimiters.

We can even access the variables passed to the template by the rendering python program via render method as stated earlier.

The delimiter is replaced with the result obtained after evaluating the expression while rendering.

Decisions

We can write the if...else statements inside the jinja template using the following syntax -

```
{% if <condition> %} <if block>
{% elif <condition2> %} <elif block>
<%else%> <else block>
<% endif %>
```

Where elif and else blocks are optional. Depending on the condition, the text inside the respective block is rendered.

Loops

Jinja provides a **for** loop very similar to the python for loop. It works on an sequence/collection of values.

It even has the range() function. Here is the syntax for the for loop in Jinja -

```
{% for <variable name> in <sequence> %}
<block inside the loops>
{% endfor %}
```



D Y PATIL
UNIVERSITY
NAVI MUMBAI

Creating static HTML pages with Jinja

Now we use some of the above-mentioned concepts to render a static html page that renders the table of a number. To render an HTML page, we just need to have the HTML markup inside the Jinja Template where required. We finally save the rendered text to an external file with a .html extension. Here is an example that creates an HTML page with the table of a given number.

Create a template with the following content and save it as "webTable.jinja" in the **jinjaTests** folder.

Create a python file to render the above template and save it as "webTable.py" in the **jinjaTests** folder.

Here are the contents of the file -

```
<!DOCTYPE html>
<html>
  <head><title>Table of {{tableOf}}</title></head>
  <body>
    <h2>Table of {{tableOf}} rendered with Jinja -</h2>
    {% for i in range(1,11) -%}
    <p>{{tableOf}} x {{i}} = {{tableOf * i}}</p>
    {%- endfor %}
  </body>
</html>
```

```
from jinja2 import Environment, FileSystemLoader

env = Environment(loader = FileSystemLoader('templates'))

template = env.get_template('webTable.jinja')

with open('renders/output.html', 'w') as f:
    print(template.render(tableOf = 5), file = f)
```

Table of 5 rendered with Jinja -

```
5 x 1 = 5
5 x 2 = 10
5 x 3 = 15
5 x 4 = 20
5 x 5 = 25
5 x 6 = 30
5 x 7 = 35
5 x 8 = 40
5 x 9 = 45
5 x 10 = 50
```



Introduction of Jinja2 Advance : -

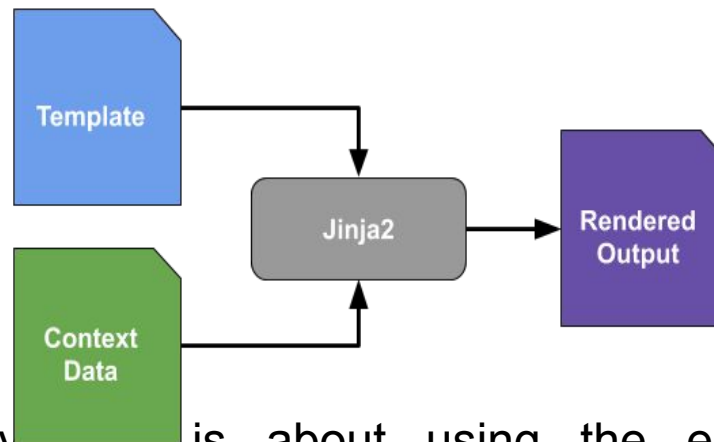
Introduction :

- Jinja2 is a modern and widely used templating engine for Python, designed to generate dynamic content by combining templates with data.
- At its core, it enables developers to embed variables, control structures, and expressions within HTML or text-based documents, thereby simplifying the process of rendering dynamic user interfaces.
- At a foundational level, Jinja2 functions as a text substitution and rendering tool, where placeholders are replaced with actual data.
- However, in advanced usage, it extends far beyond basic templating and becomes a **robust presentation** layer within application architecture.



Introduction of Jinja2 Advance : -

Separation of concerns: Business logic stays in Python, while presentation logic is handled in Jinja2 but advanced usage pushes the boundary of what templates can do without becoming messy.



In short, Jinja2 advanced is about using the engine as a scalable, maintainable, and expressive system for dynamic content generation, rather than just a simple text replacement tool.

Real-Life Example:

In modern e-commerce applications, pages like product listings, cart, and checkout share the same layout but display different data. Jinja2 simplifies this using **template inheritance**, where a base template handles common elements (header, footer) and child templates customize content. It also supports dynamic features like showing a “Sale” badge only when a discount exists and formatting prices consistently using filters. This results in a scalable, maintainable, and reusable UI without code duplication.

Syntax :-

```
{% extends "base.html" %}  
{% if product.discount %} Sale {% endif %}  
{{ price | round(2) }}
```



Elements Of Jinja2 : -

- ❖ **Template Inheritance** → Structure & reuse layouts
- ❖ **Loops** → Handle dynamic data
- ❖ **Conditions** → Control rendering logic
- ❖ **Filters** → Format output efficiently



Template Inheritance :-

Template inheritance in Jinja2 allows you to **reuse a base layout** across multiple templates.

Child templates can **extend and customize specific sections**, reducing code duplication and keeping design consistent.

- ◆ **Avoids repeating code** (header, footer, navbar).

- ◆ **Improves:** Maintainability, Consistency.

- ◆ **Advanced Use :-**

- ☐ `{{ super() }}` → keeps parent content
- ☐ `{% include %}` → reuse components
- ☐ Nested blocks supported
- ☐ Best Practice: Keep base template simple

- ◆ **Real Life Example :-**

Same layout across all pages of a website



Template Inheritance :-

Syntax :-

◆ HTML :-

```
{% extends "base.html" %}
```

```
{% block content %}
```

```
<h1>Page Content</h1>
```

```
{% endblock %}
```

Syntax Explanation :-

- {% extends %} → inherit base
- {% block %} → replace content



Loops :-

◆ Defination :-

- ❑ Jinja2 loops are control structures used to iterate over data collections like lists, tuples, and dictionaries.
- ❑ They allow templates to repeat content dynamically.
- ❑ Useful for displaying dynamic data such as menus, tables, and lists.

◆ Real Life Example :-

Displaying a list of products on a website.

◆ Advanced Features :-

❑ Advanced Loop Controls

- ❑ `loop.index` → current index
- ❑ `loop.first` / `loop.last`
- ❑ `loop.length`



Loops :-

◆ Syntax :-

```
<ul>
{% for product in products %}
    <li>{{ product.name }} - ₹{{ product.price }}</li>
{% endfor %}
</ul>
```

◆ Syntax Explanation :-

- Loops through all products.
- Displays each product dynamically.
- Reduces manual coding.
- Makes web pages dynamic and scalable.



Conditions :-

❖ Definition :

Jinja2 conditions are used to control the flow of a template based on specific conditions.
They allow templates to display different content depending on data values

☐ **Supports comparison operators** :- ==, !=, >, <, >=, <=

☐ **Logical operators** :- and, or, not

❖ Real Life Example :- Showing login status on a website

❖ Advanced Features :-

Decision Making – Show content based on logic

Dynamic UI – Different output for different users/situations

Improves User Experience – Personalized content display



Condition :

❖ **Syntax :-**

```
{% if user.is_logged_in %}  
    <p>Welcome, {{ user.name }}!</p>  
{% else %}  
    <p>Please login to continue</p>  
{% endif %}
```

❖ **Syntax Explanation :-**

- ☐ Displays personalized message if user is logged in.
- ☐ Otherwise shows login prompt.
- ☐ Improves user experience and interaction.
- ☐ Makes website dynamic and responsive.



Filters

- ◆ **Definition :-**

Filters are used to modify or format variables before displaying them. They help in formatting values like text, numbers, dates, and lists.

- ◆ **Syntax :-**

`{{ variable | filter }}`

- ◆ **Examples of Filters :-**

upper → convert to uppercase

lower → convert to lowercase

length → count items

default → set default value

join → combine list into string

- ◆ **Advanced Features :-**

- ❑ Data Formatting – Modify output (uppercase, lowercase, etc.)
- ❑ Clean Output – Handle missing or default values
- ❑ Easy Transformation – No need for extra Python code



Filter :-

❖ Syntax :-

<p>Name: {{ user.name | title }}</p>

<p>Email: {{ user.email | lower }}</p>

<p>Total Items: {{ cart | length }}</p>

❖ Syntax Explanation :-

- ☐ Formats user name properly
- ☐ Converts email to lowercase
- ☐ Counts number of items in cart
- ☐ Makes output user-friendly and professional



Setup of Jinja2 in Python (Flask) :-

Jinja2 is a template engine used with Flask to create dynamic web pages.

◆ Steps to Setup :-

- ☐ Install Python 3.12 (Add to PATH)
- ☐ Open Command Prompt (cmd)
- ☐ Install packages :- **pip install flask jinja2**
- ☐ Create project folder and app.py
- ☐ Run server :- **python app.py**

```
C:\Users\vighnesh>pip install flask jinja2
```

◆ Server Output :-

Running on http://127.0.0.1:5000

```
C:\Users\vighnesh\Desktop\flask_app>python app.py
* Serving Flask app 'app'
* Debug mode: off
WARNING: This is a development server. Do not use it in a production deployment.
* Running on http://127.0.0.1:5000
Press CTRL+C to quit
```



Code :

```
from flask import Flask, render_template_string

app = Flask(__name__)

@app.route("/")
def home():

    template = """
    <!DOCTYPE html>
    <html>
    <head>
        <title>{{ title }}</title>
        <style>
            body { font-family: Arial; margin: 40px; }
            .active { color: green; font-weight: bold; }
            .inactive { color: red; font-weight: bold; }
            .box { padding: 10px; margin: 10px; border: 1px solid #ccc; }
        </style>
    </head>

    <body>

        <h1>{{ title }}</h1>
```



Code :

```
{% for user in users %}
    <div class="box">
        {% if user.active %}
            <span class="active">{{ user.name | capitalize }} (Active)</span>
        {% else %}
            <span class="inactive">{{ user.name | capitalize }} (Inactive)</span>
        {% endif %}

        <br>

        {% if user.age >= 18 %}
            Adult
        {% else %}
            Minor
        {% endif %}
    </div>
{% endfor %}

<h3>Total Users: {{ users | length }}</h3>

</body>
</html>
```

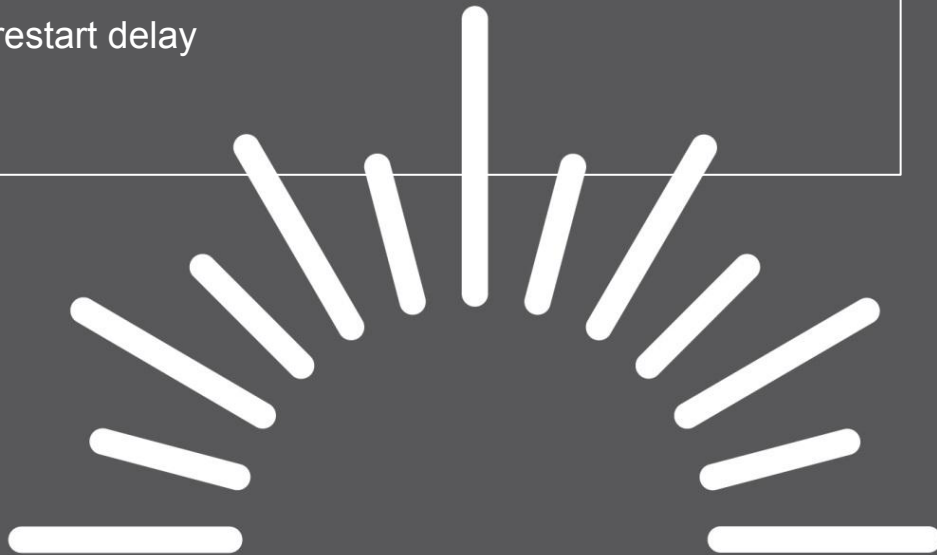


Code :

```
data = {
    "title": "Fast Jinja2 Flask App",
    "users": [
        {"name": "alice", "age": 25, "active": True},
        {"name": "bob", "age": 17, "active": False},
        {"name": "charlie", "age": 30, "active": True}
    ]
}

return render_template_string(template, **data)

if __name__ == "__main__":
    # IMPORTANT: debug=False = faster startup, no restart delay
    app.run(debug=False)
```



Output :-

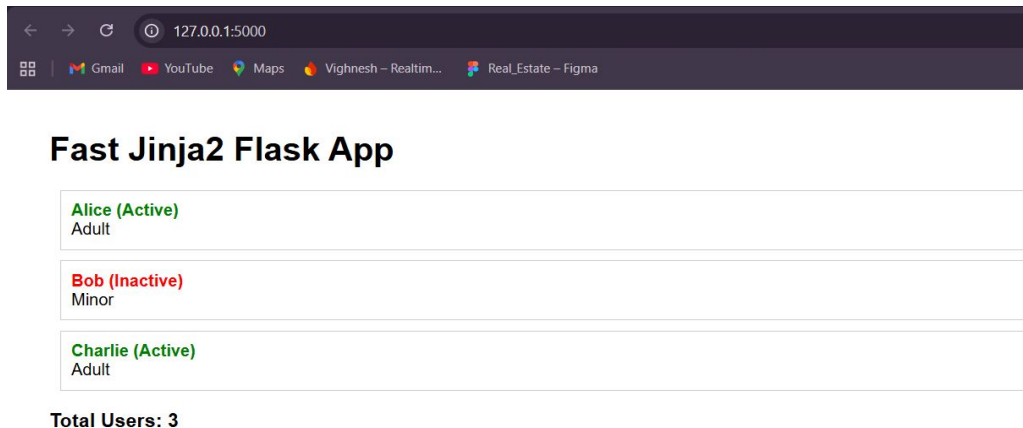
The Flask server runs successfully on localhost and renders dynamic HTML using Jinja2.

❑ **Server Output :-** Running on: <http://127.0.0.1:5000>

❑ Displays user data dynamically using:

- Loops (for)
- Conditions (if-else)

❖ Output : -



Code Explanation

❖ Code Explanation :-

- ❑ **Import Flask:** Used to create web app and render HTML.
- ❑ **Create App:** Flask(__name__) initializes application.
- ❑ **Route (/):** Connects URL to home() function.
- ❑ **Jinja2 Template:**
 - {{ }} → display data
 - {% %} → loop & conditions
- ❑ **Features Used:**
 - Loop (for users), Condition (active/age), Filter (capitalize)
- ❑ **Data:** User details passed as dictionary
- ❑ **Render:** render_template_string() shows dynamic output
- ❑ **Run Server:** Opens on <http://127.0.0.1:5000>



forms (GET/POST);

HTTP methods define how a client (browser) interacts with a server in a web application. In Flask, they are used to handle different types of requests like fetching data, sending data or updating resources.

Common HTTP Methods:

- **GET:** to request data from the server.
- **POST:** to submit data to be processed to the server.
- **PUT:** replaces the entire resource with new data. If it doesn't exist, a new one is created.
- **PATCH:** updates only specific parts of a resource without replacing the whole thing.
- **DELETE:** deletes the data on the server at a specified location.



How to handle GET & POST requests using flask in Python

To handle Requests in flask we are having a route decorator. In that, there is a methods attribute that indicates what type of request a particular function is accepting. If we keep its value empty then it by default accepts a GET request. Otherwise, we can mention methods as given in the below syntax

Syntax: `@app.route('/example', methods=['GET', 'POST'])`



GET Method

The GET method is used to request data from a server. It appends data to the URL in a name-value pair format. GET should not be used for sensitive data since URLs are visible in browser history.

To understand how the GET method works, we'll build a simple Flask app that takes a number as input, calculates its square, and displays the result. Our app will have three files:

1. **app.py** contains the Flask app code.
2. **squarenum.html** homepage where users enter a number and send it to the Flask app.
3. **answer.html** displays the calculated square of the number.



[app.py](#)

```
from flask import Flask, request, render_template
```

```
app = Flask(__name__)
```

```
@app.route('/square', methods=['GET'])
```

```
def squarenumber():
```

```
    num = request.args.get('num')
```

```
    if num is None: # No number entered, show input form
```

```
        return render_template('squarenum.html')
```

```
    elif num.strip() == "": # Empty input
```

```
        return "<h1>Invalid number. Please enter a number.</h1>"
```

```
    try:
```

```
        square = int(num) ** 2
```

```
        return render_template('answer.html', squareofnum=square,  
num=num)
```

```
    except ValueError:
```

```
        return "<h1>Invalid input. Please enter a valid  
number.</h1>"
```

```
if __name__ == '__main__':
```

```
    app.run(debug=True)
```

explanation:

- `request.args.get('num')` retrieves the number from the URL parameters.
- if **num** is **None**, the user is visiting the page for the first time otherwise an error message is displayed.
- the square of the number is calculated and passed to the **answer.html** template.



D Y PATIL
UNIVERSITY
NAVI MUMBAI

squarenum.html

```
<!DOCTYPE html>
```

```
<html lang="en">
```

```
<head>
```

```
  <meta charset="UTF-8">
```

```
  <title>Square Of Number!</title>
```

```
</head>
```

```
<body>
```

```
<h1><i> Welcome to the Maths page!</i></h1>
```

```
  <p>Logic shapes every choice of our daily  
lives.<br>
```

```
  Logical thinking enables someone to learn and  
  make decisions that affect their way of life. !</p>
```

```
  <form method="GET" action ="/square">
```

```
    Enter a number :
```

```
    <input type="text" name="num"  
id="num"></input>
```

```
    <input type="submit" name="btnnum"  
id="btnnum"></input>
```

```
  </form>
```

```
</body>
```

```
</html>
```

answer.html

```
<!DOCTYPE html>
```

```
<html lang="en">
```

```
<head>
```

```
  <meta charset="UTF-8">
```

```
  <title>Answer Page!</title>
```

```
</head>
```

```
<body>
```

```
  <h1>Keep Learning  
Maths!</h1>
```

```
  <h2>Square of number  
{{num}} is
```

```
:{{squareofnum}}</h2>
```

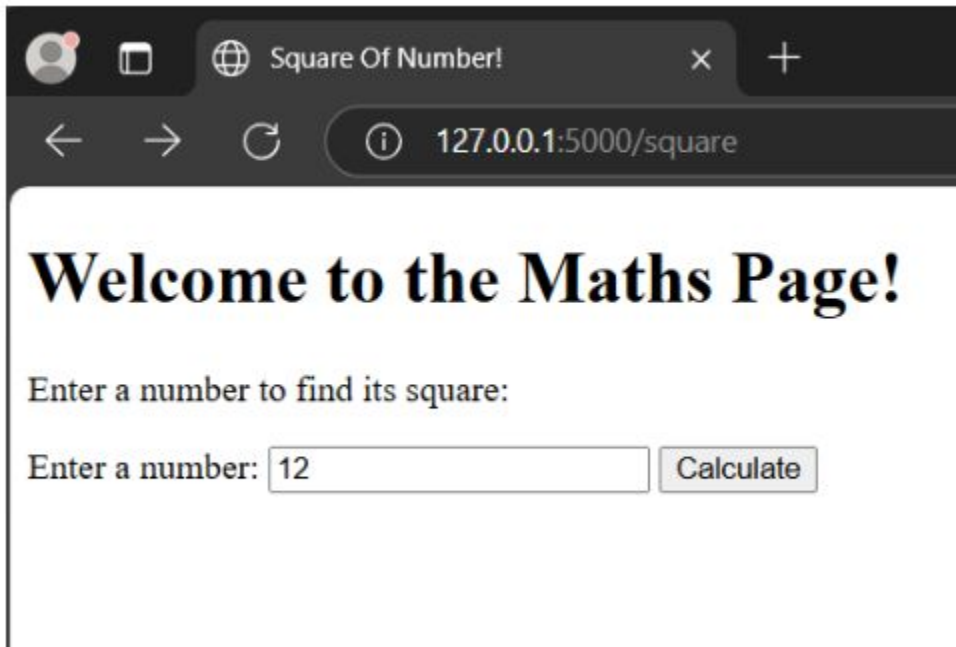
```
</body>
```

```
</html>
```

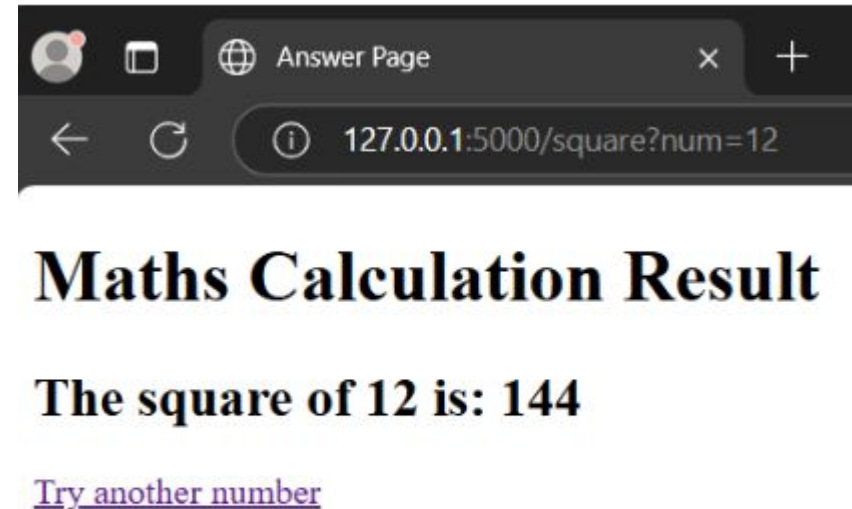


Output

Run the application and visit development server:



On clicking the Submit button, one can notice, the value entered, appended, to the URL and, the output displayed.



POST Method

The POST method is used to send data to the server for processing. Unlike GET, it does not append data to the URL. Instead, it sends data in the request body, making it a better choice for sensitive or large data.

We will modify our previous Flask app to use POST instead of GET. The app will still take a number, calculate its square and display the result. The app will have the same three files the only changes are made in the **app.py** and **squarenum.html** file.



Explanation:

[app.py](#)

```
from flask import Flask, request, render_template
```

```
app = Flask(__name__)
```

```
@app.route('/square', methods=['GET', 'POST'])
```

```
def squarenumber():
```

```
    if request.method == 'POST':
```

```
        num = request.form.get('num')
```

```
        if num.strip() == "": # Empty input
```

```
            return "<h1>Invalid number</h1>"
```

```
            square = int(num) ** 2
```

```
            return render_template('answer.html',
```

```
squareofnum=square, num=num)
```

```
            return render_template('squarenum.html')
```

```
if __name__ == '__main__':
```

```
    app.run(debug=True)
```

- **Supports both GET and POST:** The route now accepts both GET (to render the form) and POST (to process the input).
- **Data is sent in the request body:** Instead of `request.args.get('num')`, we use `request.form.get('num')` to retrieve the number from the form.
- **More secure:** Since data is not appended to the URL, it is not visible in browser history or logs.
- **Same logic for processing input:** The app still checks if the number is provided, calculates its square, and renders the result using `answer.html`.



squarenum.html: The only changes made in this code is that the form action has been changed from **GET** to **POST**

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <title>Square Of Number!</title>
</head>
<body>
  <h1>Welcome to the Maths Page!</h1>
  <p>Enter a number to find its square:</p>

  <form method="POST" action="/square">
    <label>Enter a number: </label>
    <input type="text" name="num">
    <input type="submit" value="Calculate">
  </form>
</body>
</html>
```



Output

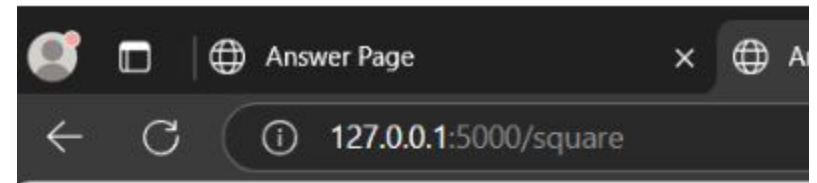


Welcome to the Maths Page!

Enter a number to find its square:

Enter a number:

On clicking the Submit button, the data is posted back, to the server, and, the result page is rendered. Please note, the value entered, is not visible in the URL now, as the method used, is POST.



Maths Calculation Result

The square of 12 is: 144

[Try another number](#)



integrate MySQL CRUD via mysql-connector;

Operation	Description
Create	Insert new record
Read	Fetch/display data
Update	Modify existing data
Delete	Remove data

Working with MySQL Workbench

Step-1: Install MySQL workbench. Link to install : <https://dev.mysql.com/downloads/workbench/> Know more about it : <https://www.mysql.com/products/workbench/>

Step-2: Install 'mysqlbd' module in your venv.

```
pip install flask-mysqldb
```

Step 3: Open MySQL workbench.

Step 4: Write the following code. The above SQL statement will create our database **geekprofile** with the table **accounts**.

Step-5: Execute the query.



Creating Project

Step 1: [Create an environment](#), and [install Flask](#)

Step 2: Create an empty folder 'geeksprofile'.

Step 3: Now open your code editor and open this 'geeksprofile' folder.

Step 4: Create 'app.py' folder and write the code given below.

<https://www.geeksforgeeks.org/python/profile-application-using-python-flask-and-mysql/>

follow weblink for code

Step 5: Create the folder 'templates'. create the files 'index.html', 'display.html', 'update.html', 'login.html', and 'register.html' inside the 'templates' folder.

Step 6: Open the 'login.html' file and write the code given below. In 'login.html', we have two fields i.e. username and password. When a user enters the correct username and password, it will route you to the index page otherwise 'Incorrect username/password' is displayed.

Step 7: Open 'register.html' file and write the code given below. In 'register.html', we have nine fields i.e. username, password, email, organisation, address, city, state, country, postal code. When user enters all the information, it stored the data in the database and 'Registration successful' is displayed.

Step 8: Open 'index.html' file and write the code given below. When a user logs in successfully, this page is displayed, and 'Logged in successful' is displayed.

Step 9: Open 'display.html' file and write the code given below. Here, the user information stored in database are displayed.

Step 10: Open 'update.html' file and write the code given below. The user can update his/her data which also updates the database.



Run the Project

Step-1: Run the server.

Step-2: Browse the URL 'localhost:5000'.

Step-3: The output web page will be displayed.

Testing of the Application

Step-1: If you are new user, go to sign up page and fill the details.

Step-2: After registration, go to login page. Enter your username and password and sign in.

Step-3: If your login is successful, you will be moved to index page and your name will be displayed.

Step-4: You can view your profile details in display page and also you can update your details in update page.

Error	Cause
Access denied	Wrong MySQL password
Module not found	mysql-connector not installed
Table not found	DB/table not created
Changes not saved	Missing commit()



simple app (TODO list/student DB viewer).

- There are many frameworks that allow building your webpage using Python, like Django, flask, etc. Flask is a web application framework written in Python.
- Flask is based on WSGI(Web Server Gateway Interface) toolkit and Jinja2 template engine.
- Its modules and libraries that help the developer to write applications without writing the low-level codes such as protocols, thread management, etc.
- **Installation:** `pip install Flask`

Basic setup : Step 1: First make basic folders

```
mkdir app && cd app && mkdir static && mkdir templates
```

Step 2: Make some basic python files to write the code and name it as you desire. **Step 3:** Run the below command to start the server

```
touch run.py the app
```

Step 4: Change directory to *app*-

```
cd app
```

Step 5: create *models.py* for database, *routes.py* for urls/views and *__init__.py* file to package our app

```
touch models.py routes.py __init__.py
```

Step 6: Goto *templates/* directory and create *index.html* file

```
cd templates && touch index.html
```

Step 7: Goto *static/* directory and create *main.css*

```
cd static && touch main.css
```



Create a TODO List Web App using Flask (add, view, delete tasks). (L5 – Evaluate)

1. Introduction

A **TODO List Web Application** allows users to manage daily tasks by:

- Adding new tasks
- Viewing all tasks
- Deleting completed tasks

Flask (a Python web framework) is used to build the application with routing and templates.

2. Requirements

- Python
- Flask library
- Web browser

Install Flask:

pip install flask

3. Application Design

- Tasks are stored in a **list (temporary storage)**
- Flask handles HTTP requests
- HTML templates display data

4. Flask Application Code

```
from flask import Flask, render_template, request, redirect

app = Flask(__name__)

tasks = [] # list to store tasks

# View tasks
@app.route('/')
def index():
    return render_template('index.html', tasks=tasks)

# Add task
@app.route('/add', methods=['POST'])
def add():
    task = request.form.get('task')
    if task:
        tasks.append(task)
    return redirect('/')

# Delete task
@app.route('/delete/<int:id>')
def delete(id):
    if 0 <= id < len(tasks):
        tasks.pop(id)
    return redirect('/')

if __name__ == '__main__':
    app.run(debug=True)
```

5. HTML Template (index.html)

```
<!DOCTYPE html>
<html>
<head>
    <title>TODO App</title>
</head>
<body>

<h2>TODO List</h2>

<form method="POST" action="/add">
    <input type="text" name="task" placeholder="Enter task">
    <input type="submit" value="Add">
</form>

<ul>
    {% for t in tasks %}
    <li>
        {{ t }}
        <a href="/delete/{{ loop.index0 }}">Delete</a>
    </li>
    {% endfor %}
</ul>

</body>
</html>
```



6. Working

1. User enters a task in the form
 2. Task is sent using POST request
 3. Flask stores task in list
 4. All tasks are displayed on homepage
 5. User can delete a task using its index
-

7. Output

- User can:
 - Add tasks
 - View list of tasks
 - Delete tasks

8. Advantages

- Simple and lightweight
- Easy to implement using Flask
- Demonstrates CRUD concepts

9. Limitations

- Data is not stored permanently (lost on restart)
- No database integration

10. Conclusion

The TODO List app demonstrates the use of Flask for handling routes, form data, and dynamic content rendering. It is a basic example of a web application implementing **Create, Read, and Delete operations**.



Deploy demo (Heroku/local)

Explain step-by-step deployment of Flask application on local server and on Heroku.

<https://www.geeksforgeeks.org/python/deploy-python-flask-app-on-heroku/>

What is Deployment?

Deployment means making a web application available so users can access it:

- Locally → on your own system
- Cloud → on a public server (internet access)

Flask is a web application framework written in Python.

Flask is based on the Werkzeug WSGI toolkit and Jinja2 template engine. Both are Pocco projects.

To demonstrate this, we are first going to create a sample application for a better understanding of the process.

Prerequisites

- Python
- pip
- Heroku CLI
- Git

Deploying Flask App on Heroku

Let's create a simple flask application first and then it can be deployed to heroku. Create a folder named "**eflask**" and open the command line and cd inside the "**eflask**" directory.

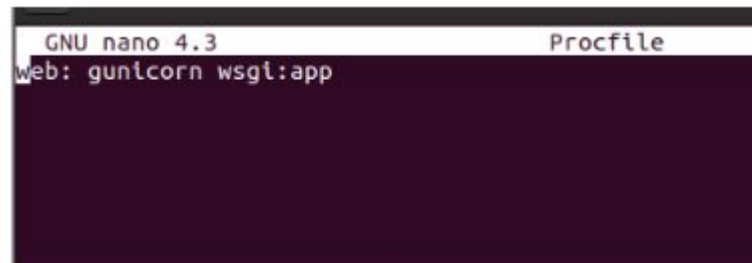
Follow the following steps to create the sample application for this tutorial.

STEP 1 : Create a virtual environment with pipenv and install **Flask** and **Gunicorn** .

```
$ pipenv install flask gunicorn
```

STEP 2 : Create a "**Procfile**" and write the following code.

```
$ touch Procfile
```



```
GNU nano 4.3 Procfile
web: gunicorn wsgi:app
```

STEP 3 : Create "**runtime.txt**" and write the following code.

```
$ touch runtime.txt
```



```
GNU nano 4.3 runtime.txt
python-3.7.5
```

STEP 4 : Create a folder named "**app**" and enter the folder.

```
$ mkdir app
$ cd app
```

STEP 5 : Create a python file, "**main.py**" and enter the sample code.

```
touch main.py
```

What is Heroku?

Heroku is a **Platform as a Service (PaaS)** used to deploy web apps easily without managing servers.

Advantages of Heroku

- Easy deployment
- No server management
- Free tier available (limited)
- Supports multiple languages

Limitations

- Free apps may **sleep after inactivity**
- Limited resources
- Requires internet connection

Deployment is the final step in web development.

Local deployment is useful for testing, while cloud platforms like Heroku make applications accessible globally.



LOCAL DEPLOYMENT

Steps for Local Deployment

Step 1: Create Project Structure

```
todo-app/  
├──  
├── app.py  
├── templates/  
│   └── index.html
```

Step 2: Install Flask

```
pip install flask
```

Step 3: Run Flask App

```
python app.py
```

Step 4: Access in Browser

```
http://127.0.0.1:5000/
```

Key Points (Local)

- Flask runs on **development server**
- debug=True → auto reload + error messages
- Not suitable for production
- Good for testing and teaching



Explain Flask architecture and request-response cycle with diagram.

<https://testdriven.io/blog/how-are-requests-processed-in-flask/>

1. Introduction to Flask Architecture

Flask is a **micro web framework** written in Python. It is called “micro” because it provides only the core components required for web development, such as:

- Routing
- Request handling
- Template rendering

Flask follows a **modular architecture** and is based on the **WSGI (Web Server Gateway Interface)** standard, which allows communication between web servers and Python applications.

2. Components of Flask Architecture

1. Client (Browser)

- Sends HTTP requests (GET/POST) to the server
- Displays the response received

2. Web Server (WSGI Server)

- Example: Werkzeug (default Flask server)
- Receives requests from client
- Passes them to Flask application

3. Flask Application

- Core part where logic is written
- Handles routing and request processing

4. Routing System

- Defined using `@app.route()`
- Maps URL to a Python function

5. View Functions

- Functions that process requests and return responses

6. Templates (Jinja2)

- Used to generate dynamic HTML pages

7. Database Layer (Optional)

- MySQL / SQLite used for CRUD operations

3. Request–Response Cycle in Flask

The **request-response cycle** defines how a user request is processed and how a response is returned.

Step-by-Step Process

Step 1: Client Request

- User enters URL (e.g., `http://localhost:5000/home`)
- Browser sends an HTTP request (GET/POST)

Step 2: Request Received by Server

- Web server (Werkzeug) receives the request
- Forwards it to Flask application

Step 3: URL Routing

- Flask checks URL in routing table
- Matches it with `@app.route()`

```
@app.route('/home')
```

```
def home():
```

```
    return "Welcome"
```

Step 4: View Function Execution

- Corresponding function is executed
- Business logic is processed
- May interact with database

Step 5: Template Rendering

- If HTML page is needed, Flask uses Jinja2
- ```
return render_template("home.html", name="Prachi")
```

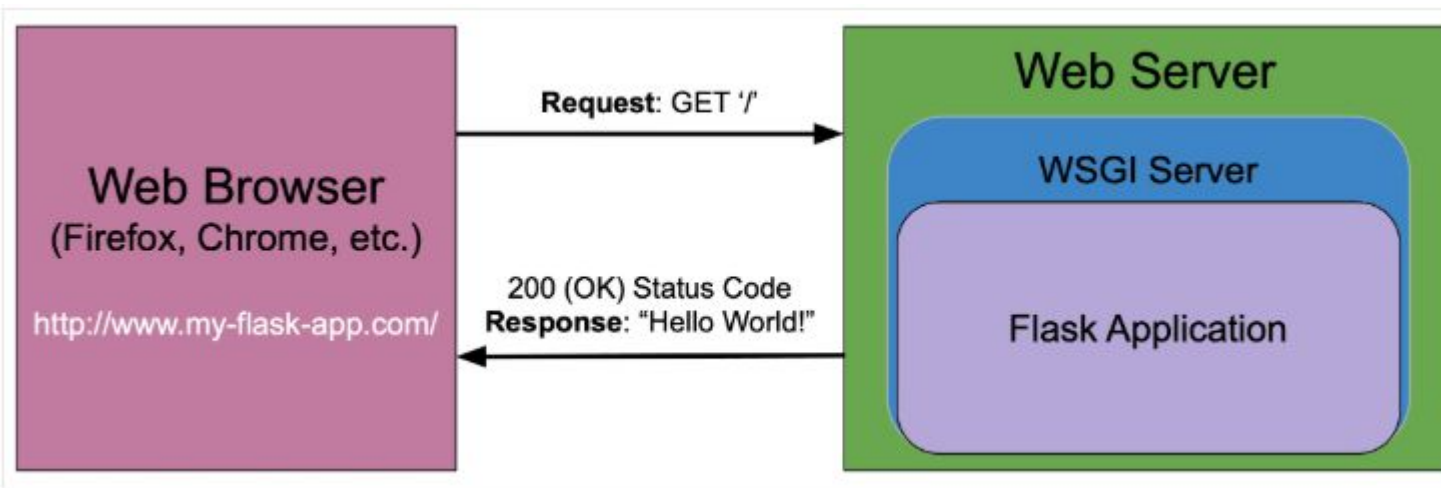
#### Step 6: Response Creation

- Flask creates HTTP response (HTML/JSON)

#### Step 7: Response Sent to Client

- Server sends response back
- Browser displays the output





## 5. Example of Request–Response Cycle

```
from flask import Flask, render_template
app = Flask(__name__)
```

```
@app.route('/')
def index():
 return "Hello, World!"
```

```
@app.route('/user/<name>')
def user(name):
 return render_template('user.html', name=name)
```

```
app.run(debug=True)
```

### Flow:

1. User visits `/user/Prachi`
2. Flask matches route
3. Executes `user()` function
4. Passes data to template
5. Returns HTML response

## 6. Key Features of Flask Architecture

- Lightweight and flexible
- Uses built-in development server
- Supports RESTful request handling
- Easy integration with databases
- Uses Jinja2 for dynamic content

## 7. Conclusion

Flask follows a **simple and efficient architecture** based on the **request-response cycle**, where:

- Client sends request
- Flask processes it using routing and view functions
- Response is generated and returned

This simplicity makes Flask ideal for:

- Small web apps
- APIs
- Student projects (like TODO apps, CRUD apps)



**D Y PATIL**  
UNIVERSITY  
NAVI MUMBAI

# Develop a Flask application demonstrating routing, templates, and dynamic content.

---

## Flask Application Code (app.py)

```
from flask import Flask, render_template

app = Flask(__name__)

Home route
@app.route('/')
def home():
 return render_template('index.html')

Dynamic route with variable
@app.route('/user/<name>')
def user(name):
 return render_template('user.html',
username=name)

Route with dynamic list
@app.route('/items')
def items():
 item_list = ["Laptop", "Mobile", "Tablet",
"Headphones"]
 return render_template('items.html',
items=item_list)

if __name__ == '__main__':
 app.run(debug=True)
```

## Templates (Jinja2)

### index.html

```
<!DOCTYPE html>
<html>
<head>
 <title>Home</title>
</head>
<body>
 <h1>Welcome to Flask App</h1>
 <p>Go to User Page</p>
 <p>View Items</p>
</body>
</html>
```

### user.html

```
<!DOCTYPE html>
<html>
<head>
 <title>User Page</title>
</head>
<body>
 <h1>Hello, {{ username }}!</h1>
 <p>This is dynamic content using Jinja2.</p>
</body>
</html>
```

### items.html

```
<!DOCTYPE html>
<html>
<head>
 <title>Items</title>
</head>
<body>
 <h1>Item List</h1>

 {% for item in items %}
 {{ item }}
 {% endfor %}

</body>
</html>
```



---

## Explanation

### Routing

- Defined using `@app.route()`
  - Maps URL → function
- Example:

```
@app.route('/user/<name>')
```

### Templates (Jinja2)

- Used to generate dynamic HTML
- Variables: `{{ username }}`
- Loops:

```
{% for item in items %}
```

### Dynamic Content

- Data passed from Python → HTML
- Example:

```
return render_template('items.html', items=item_list)
```

## Conclusion

This application demonstrates how Flask:

- Handles routes
- Uses templates
- Displays dynamic content

It forms the base for advanced apps like:

- Student database systems
- TODO applications



# Design a Flask application to accept user input via form and display result using Jinja2.

## Objective

To design a Flask web application that:

- Accepts user input through an HTML form
- Processes data using Flask
- Displays the result dynamically using Jinja2 templates

### result.html

```
<!DOCTYPE html>
<html>
<head>
 <title>Result</title>
</head>
<body>
 <h2>Submitted Details</h2>

 <p>Name: {{ name }}</p>
 <p>Age: {{ age }}</p>
 <p>Status: {{ status }}</p>

 Go Back
</body>
</html>
```

## Flask Application Code (app.py)

```
from flask import Flask, render_template, request

app = Flask(__name__)

Route to display form
@app.route('/')
def form():
 return render_template('form.html')

Route to handle form submission
@app.route('/submit', methods=['POST'])
def submit():
 name = request.form['name']
 age = request.form['age']

 # Simple processing logic
 if int(age) >= 18:
 status = "Adult"
 else:
 status = "Minor"

 return render_template('result.html', name=name,
 age=age, status=status)

if __name__ == '__main__':
 app.run(debug=True)
```

## HTML Templates

### form.html

```
<!DOCTYPE html>
<html>
<head>
 <title>User Form</title>
</head>
<body>
 <h2>User Input Form</h2>

 <form action="/submit" method="POST">
 Name: <input type="text" name="name"
 required>

 Age: <input type="number" name="age"
 required>

 <input type="submit" value="Submit">
 </form>

</body>
</html>
```



## Explanation

### Form Handling (POST Method)

- Form sends data using:

```
<form method="POST">
```

- Data accessed in Flask:

```
request.form['name']
```

### Routing

- `/` → Displays form
- `/submit` → Processes data

### Dynamic Content (Jinja2)

- Variables displayed using:

```
{{ name }}
{{ status }}
```

### Processing Logic

- Age-based classification:

```
if int(age) >= 18:
```

## Request–Response Flow

User → Form (HTML) → Flask (/submit) → Process Data → Jinja2 Template → Output Page

### Sample Output

Input:

- Name: Prachi
- Age: 22

Output:

Name: Prachi  
Age: 22  
Status: Adult

### Conclusion

This application demonstrates how Flask:

- Accepts user input via forms
- Processes it on server-side
- Displays dynamic results using templates

This forms the base for:

- Login systems
- Registration forms
- Student data processing apps



**D Y PATIL**  
UNIVERSITY  
NAVI MUMBAI

# Explain GET vs POST methods with implementation in Flask forms. (L4 – Analyze)

## 1. Introduction

HTTP methods define how data is sent between client (browser) and server.

The two most commonly used methods in web applications are:

- GET → used to retrieve data
- POST → used to send data securely to the server

Flask supports both methods for handling form data.

## 2. GET Method

Definition

GET method sends data as URL parameters (query string).

Example:

<http://example.com/form?name=Prachi&age=20>

### Characteristics

- Data visible in URL
- Limited data size
- Can be bookmarked
- Faster, used for fetching data
- Less secure

## Flask Implementation (GET)

### app.py

```
from flask import Flask, request, render_template
```

```
app = Flask(__name__)
```

```
@app.route('/form', methods=['GET'])
```

```
def form():
```

```
 name = request.args.get('name')
```

```
 return render_template('form.html', name=name)
```

```
if __name__ == '__main__':
```

```
 app.run(debug=True)
```

### form.html

```
<form method="GET" action="/form">
```

```
 Name: <input type="text" name="name">
```

```
 <input type="submit">
```

```
</form>
```

```
{% if name %}
```

```
<p>Hello {{ name }}</p>
```

```
{% endif %}
```

## Working

1. User enters data
2. Data appended in URL
3. Flask retrieves using `request.args.get()`
4. Output displayed



### 3. POST Method

## Flask Implementation (POST)

#### Definition

POST method sends data in the **HTTP request body**, not in URL.

#### Characteristics

- Data not visible in URL
- Can send large data
- More secure than GET
- Cannot be bookmarked
- Used for form submission, login, database operations

#### app.py

```
from flask import Flask, request, render_template
```

```
app = Flask(__name__)
```

```
@app.route('/submit', methods=['GET', 'POST'])
```

```
def submit():
```

```
 name = None
```

```
 if request.method == 'POST':
```

```
 name = request.form.get('name')
```

```
 return render_template('submit.html', name=name)
```

```
if __name__ == '__main__':
```

```
 app.run(debug=True)
```

#### submit.html

```
<form method="POST" action="/submit">
```

```
 Name: <input type="text" name="name">
```

```
 <input type="submit">
```

```
</form>
```

```
{% if name %}
```

```
<p>Hello {{ name }}</p>
```

```
{% endif %}
```

#### Working

1. User submits form
2. Data sent in request body
3. Flask reads using `request.form.get()`
4. Response generated



Feature	GET	POST
Data Location	URL	Request Body
Security	Low	Higher
Data Size	Limited	Large
Bookmarking	Yes	No
Usage	Fetch data	Submit data
Flask Access	<code>request.args</code>	<code>request.form</code>

#### Analysis (Important for L4 Level)

- **GET is suitable** for search, filtering, and retrieving data because it is fast and bookmarkable.
- **POST is suitable** for sensitive operations like login, registration, and database updates because data is hidden from URL.
- Using GET for sensitive data is insecure.
- POST ensures better data handling and supports larger payloads.

#### Conclusion

- Both GET and POST methods are essential in web development.
- Flask provides simple mechanisms (`request.args` and `request.form`) to handle them efficiently.
- Choosing the correct method improves security, performance, and usability of the application.



# Thank You



**D Y PATIL**  
UNIVERSITY  
NAVI MUMBAI